

A Software-Based Alternative to Physical Logic Circuit Training Kits

A Python-driven simulation environment, extended into a drag-and-drop web application (v2.0)

Abstract

Logic circuit education is commonly taught using physical training kits, which restrict the number of components available and therefore the complexity of the circuits students can design and test. This project presents a software alternative: a Python-based simulation environment built around a custom, human-readable syntax for describing logic gates, which was later re-implemented as a drag-and-drop web application (v2.0) with real-time, color-coded signal visualization. Development began in the 3rd grade (2024) and reached a stable, working state with version v1.2. The system removes the hardware limitation on gate count, lets users chain gates into arbitrarily complex topologies, and provides immediate visual feedback for debugging. A live demo of the web version is available online.

1. Introduction and Problem Statement

Physical logic-circuit training kits, such as the one shown in Figure 1, are widely used in laboratory settings to teach digital logic. However, they impose a hard limit on the number of gates and connections a student can use at once, which constrains the complexity of the circuits that can be built and makes experimentation cumbersome. The goal of this project was to remove this hardware constraint and make logic circuit education more accessible and practical by replacing the physical kit with software.

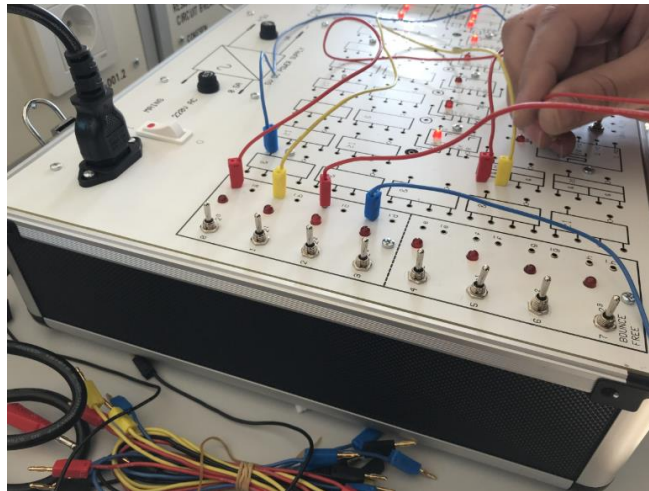


Figure 1. Physical logic-circuit training kit used in the laboratory.

To address this problem, a simulation environment running entirely on Python was developed, and later, in version V2.0, its design was strengthened and moved to the web. Unlike the physical kit, the resulting system allows the user to create an unlimited number of logic gates and test them freely. Development began while the author was in the 3rd grade (2024); after overcoming the technical difficulties involved in translating the design into code, the system was made stable and functional with version v1.2.

2. Methodology: A Custom Syntax for Logic Gates (v1.2)

In the Python-based simulation environment, a custom definition language (syntax) was created to convert visual circuit diagrams into an algorithmic structure. The basic principle of this structure is to define the

operation of each logic gate in the format **Output—GATE=Input1,Input2**. For example, a simple AND gate operation is expressed as shown in Figure 2.

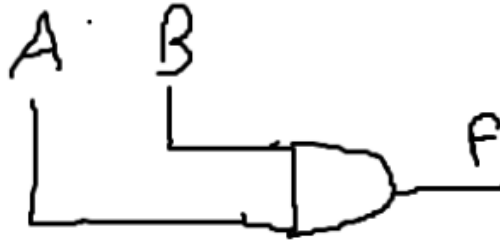


Figure 2. A simple AND gate.

For the circuit in Figure 2, this is written in code as “F—AND=A,B”. This expression indicates that the F output is obtained by subjecting inputs A and B to the AND operation.

2.1 Chaining Capability and Complex Circuit Design

The most critical capability of the system is its modular processing structure: the intermediate output obtained from one logic gate can be defined as the input of another gate in a subsequent step. This allows the user to build complex circuit topologies from an unlimited number of gates.

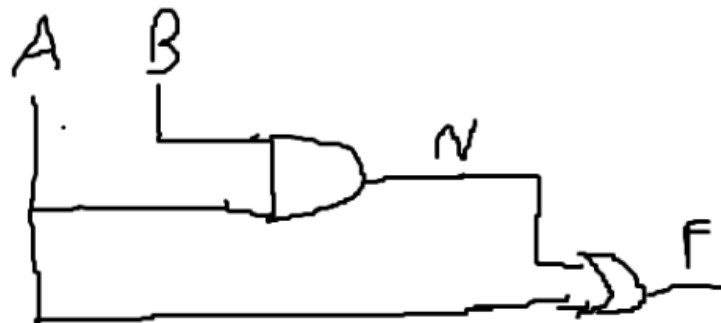


Figure 3. Chaining two gates: $N = A \text{ AND } B$, then $F = N \text{ OR } A$.

As illustrated in Figure 3:

1. $N\text{—AND}=A,B$: in the first step, A and B are processed to obtain the intermediate value N.
2. $F\text{—OR}=N,A$: the value N is combined with input A using an OR operation to produce the final result F.

With this method, layered logic circuits can be modeled easily in text form.

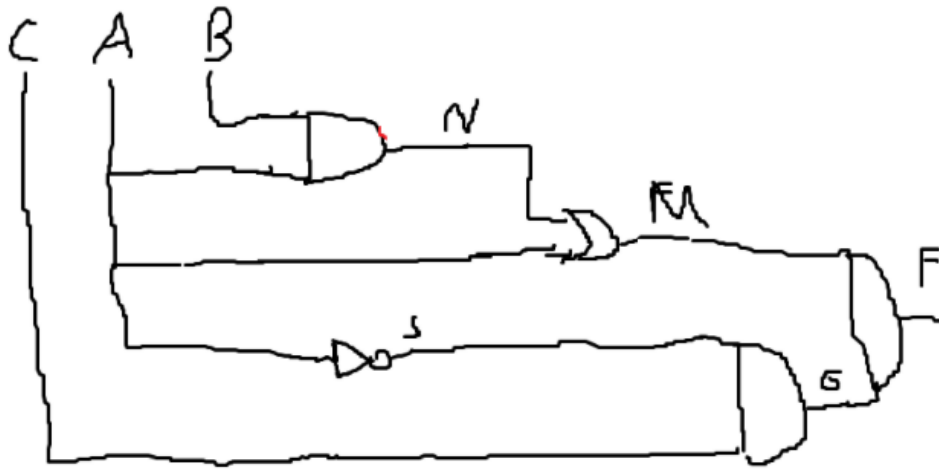


Figure 4. A more complex, five-gate circuit (inputs A, B, C; output F).

The more comprehensive circuit shown in Figure 4 was translated into code with the following steps:

- N—AND=A,B (AND operation of A and B)
- M—OR=N,A (OR operation of N and A)
- S—NOT=A (inverse of A — NOT operation)
- G—AND=C,S (AND operation of C and S)
- F—AND=M,G (AND operation of M and G, giving the final result)

3. Implementation and Simulation (v1.2)

The algorithm parses the defined command set and identifies which independent variables (e.g., A, B, C) appear in the circuit. Once the user assigns binary truth values (0 or 1) to these variables, the system simulates all intermediate operations (N, M, S, G) and computes the final F result. The processing steps, user interaction, and output analysis carried out in the Python simulation are presented below.

3.1 Circuit Definition and Variable Assignment

Using the syntax described above, the circuit structure is defined in the system (Figure 5). After the definition step, the algorithm analyzes the logic gates in the circuit and requests the necessary input variables from the user.

```

Kaç adet kod yazacaksınız?
= 5
1. Satır1 Yaz
> N--AND=A,B
2. Satır1 Yaz
> M--OR=N,A
3. Satır1 Yaz
> S--DEGIL=A
4. Satır1 Yaz
> G--AND=C,S
5. Satır1 Yaz
> F--AND=M,G
Kaç adet Girdi yazacaksınız?
= 3
Girdi ne? [Orn: A,B..]
> C
Girdi ne? [Orn: A,B..]
> A
Girdi ne? [Orn: A,B..]
> B

```

Figure 5. Entering the gate definitions in the command-line interface.

The input variables (A, B, C, etc.) expected by the system are then listed and presented for user confirmation (Figure 6). This step lets the user see, in complex circuits, exactly which ports require input data.

```

Kaç adet kod yazacaksınız?
= 5
1. Satır1 Yaz
> N--AND=A,B
2. Satır1 Yaz
> M--OR=N,A
3. Satır1 Yaz
> S--DEGIL=A
4. Satır1 Yaz
> G--AND=C,S
5. Satır1 Yaz
> F--AND=M,G
Kaç adet Girdi yazacaksınız?
= 3

```

Figure 6. Confirming the required input variables (C, A, B).

3.2 Simulation Execution and Results

Once the input variables are set, binary (0 or 1) truth values are assigned to each one. The simulation engine processes these values through the chain of logic gates to compute the final F result. As shown in Figure 7, for the entered values (C=1, A=0, B=1), the system output was found to be F=0.

```

C için deęer girin (0 veya 1): 1
A için deęer girin (0 veya 1): 0
B için deęer girin (0 veya 1): 1
0
0
1
1
0
Sonuç: F = 0
Yeni deęer girmek istiyor musunuz? [1: Evet, 0: Hayır]
=

```

Figure 7. Simulation run: inputs C=1, A=0, B=1 yield output F=0.

3.3 User Experience and Continuous Testing

To improve the user experience during analysis, a “Continuous Test Loop” was implemented: after running one calculation, the user can test different scenarios by simply changing the input values, without redefining the circuit from scratch. Figure 8 shows the circuit being re-tested with a new set of inputs (C=1, A=0, B=1) and the corresponding signal values annotated on each wire.

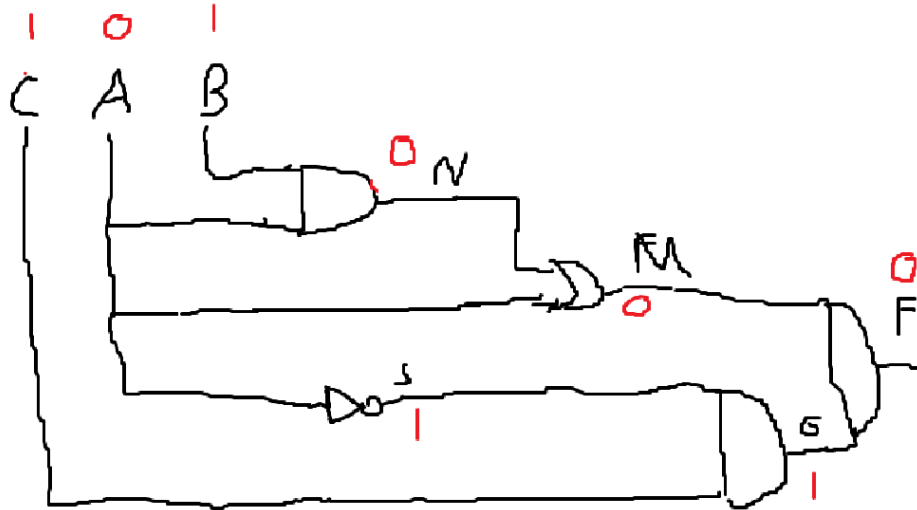


Figure 8. Re-testing the circuit with new input values; intermediate signal values are annotated.

4. Web-Based Implementation (v2.0)

The original Python version (v1.2) runs from the command line. To increase accessibility, the next stage of the project (v2.0) re-implements the simulator as a web-based environment with a visual, drag-and-drop interface. Figure 9 shows the current version: gates are placed on the canvas by dragging them from the component panel on the left.

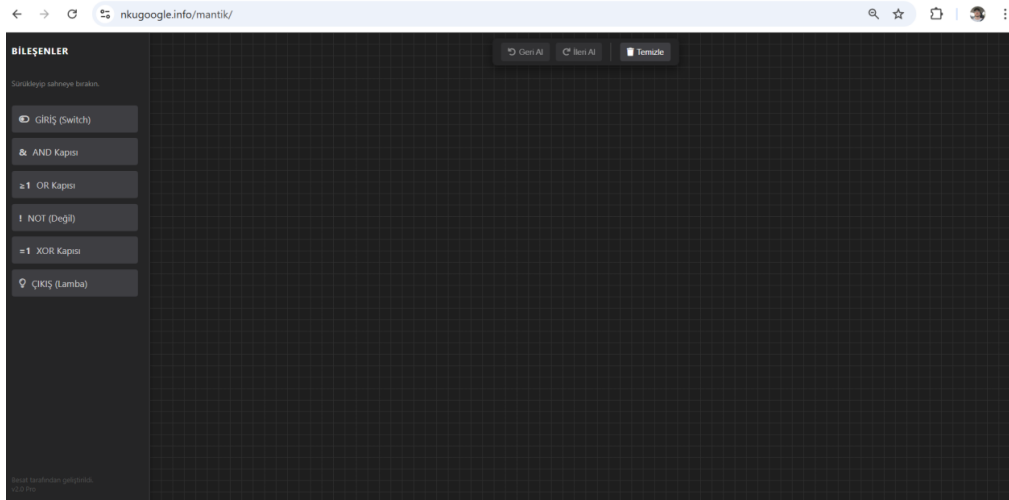


Figure 9. The web-based interface (v2.0) with the component panel on the left.

After a gate is placed, its inputs and outputs are wired together by pressing and holding a connection point and dragging it to another gate's terminal, as shown in Figure 10.

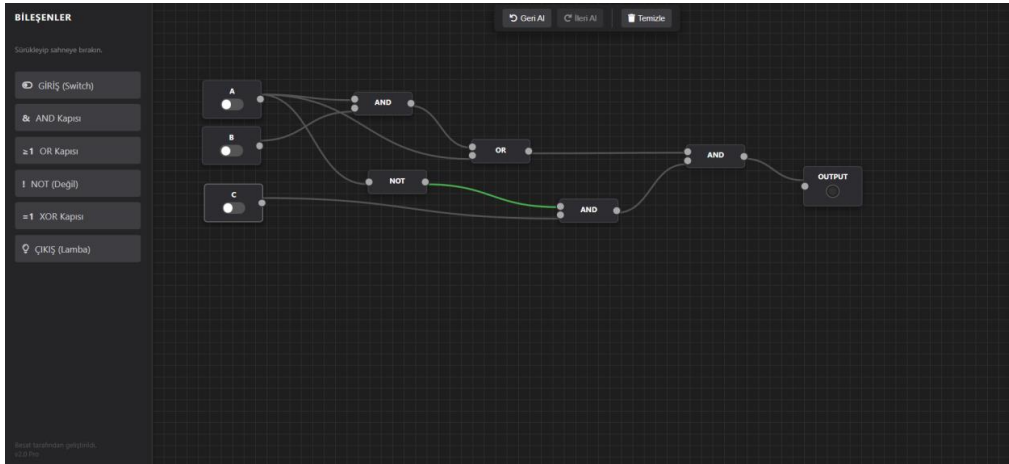


Figure 10. Wiring gate inputs and outputs together by drag-and-drop.

To let users follow the logic flow more easily, the web interface uses a color-coded feedback mechanism: green represents the logical '1' (High) level, while black (or dim) represents the logical '0' (Low) level. The circuit structure previously verified in Python was re-built in the web interface, and the same test conditions were applied, with inputs set to $A=0$, $B=1$, $C=1$ for consistency with the earlier experiment (Figure 11).

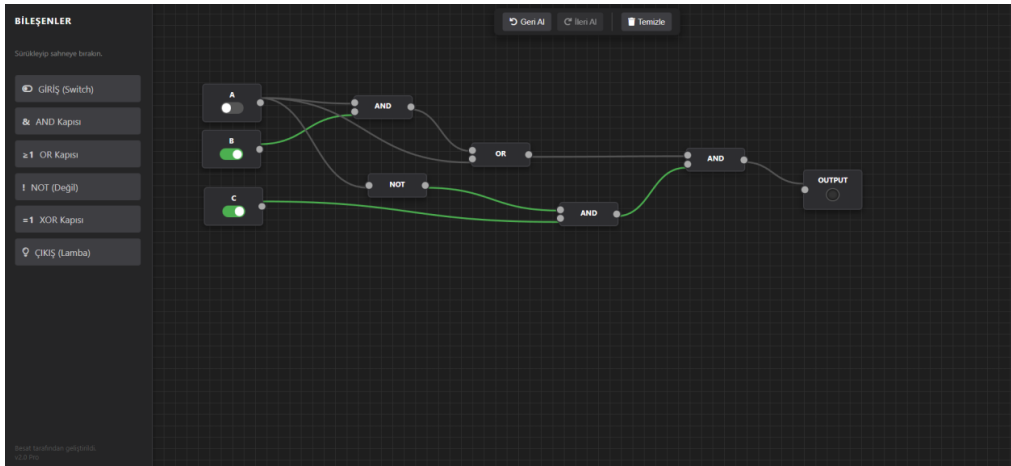


Figure 11. Web-based simulation with A=0, B=1, C=1; output remains 0 (black).

The result in Figure 11 shows the final output is inactive (black/0), confirming correct operation. By varying the inputs, the propagation of 1s and 0s through the wires can be followed live, and for this particular circuit the output is always 0, so the lamp never lights up. The system's key strength — real-time visual signal tracking — makes it possible to see instantly which lines carry '1' (green) and which carry '0' (black) at every intermediate layer, which greatly simplifies debugging faulty designs.

4.1 Dynamic Circuit Modification

With the original circuit, no combination of inputs could produce an output of '1' (every possibility was tried). This exposed a further advantage of the software: gates can be added on the fly. To confirm the system's flexibility, a NOT gate was inserted at the end of the circuit — added instantly via drag-and-drop — specifically to bring the output to the logical '1' level (Figure 12). Following this modification, the signal was inverted and the circuit output became active (green/1).



Figure 12. Adding a NOT gate at runtime inverts the output to '1' (green).

5. Results and Discussion

This work presents a highly accessible simulation ecosystem for logic circuit education and design. The project began with a Python-based syntax architecture (v1.2) that eliminates hardware limitations, and evolved into a web-based visual interface (v2.0) centered on user experience. The main achievements of the developed system are as follows:

1. **Unlimited design freedom:** removing the gate-count limitation of physical kits enables the design of arbitrarily complex circuits.
2. **Visualization:** color-coded signal tracking (green/black) helps students and enthusiasts grasp abstract logic operations more concretely.
3. **Interaction:** the drag-and-drop interface and instant re-testing encourage learning through trial and error.

6. Conclusion and Future Work

By carrying the educational foundation of physical kits such as the one in Figure 1 into the digital world, this project represents a step toward making logic circuit education more democratic, accessible, and understandable. Future work aims to extend the system to support more complex integrated circuits and to adapt it for mobile platforms.

A live version of the web application is available for testing at: <https://nkugoogle.info/mantik/>

Appendix: Mobile Interface

Figure 13 shows the responsive mobile layout of the web application, toggling an input switch from off (output unlit) to on (output lit, via the NOT gate).

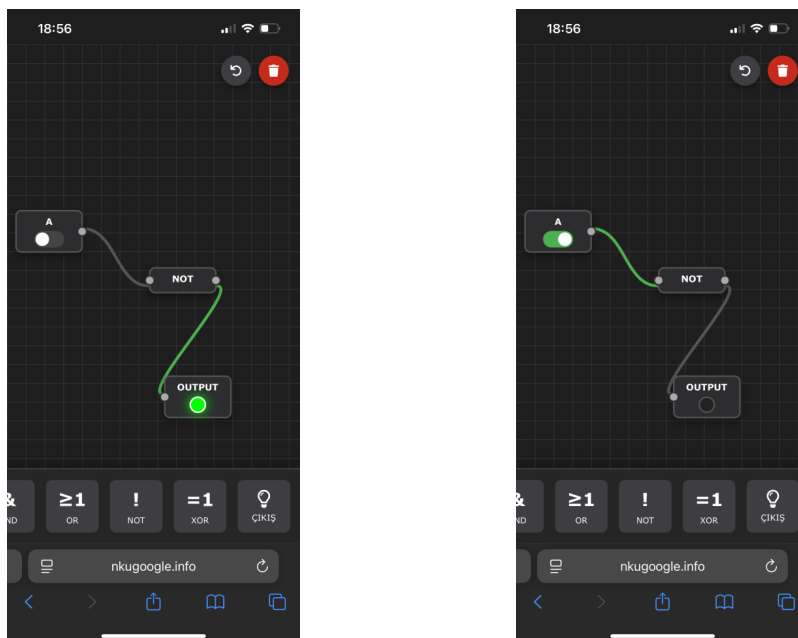


Figure 13. Mobile interface: input off (left) and input on, output active via NOT gate (right).